训练神经网络

Module 3 · Topic 2 — 知识点完整整理

ARTI2001 Machine Learning · 2026 Semester 2

涵盖 **Topic 2 四课**

sigmoid / tanh / ReLU 及导数 · **饱和与梯度行为**

MSE 与交叉熵 · **四种任务的输出层与损失选择**

隐藏层权重梯度的完整反向传播手算

batch · epoch · **自动微分（GradientTape 三问）**

M3 测验：Week 13（2026-11-09 ~ 11-15）· 110 分钟 · 开卷 · 20 选择×3 + 4 大题×10

目录

M3 T2 第 1 课 · 激活函数

· 0. 这一课的主旨（页面开宗明义的一句）
· 1. ★★★ 关键区分：输出值 vs 导数
· 2. ★★★ 四个激活函数（必须会默写这张表）
· 3. ★★ 三个点上的数值表（最可能直接出计算题）
· 4. ★★ 饱和与「死亡 ReLU」
· 5. ★★ 非线性的概率输出，笔直的决策边界（T1 讲过，这里给了具体数字）
· 6. ★★★ 输出层怎么选（M3 考纲逐条点名的内容）
· 7. 总结自测
· ★ 速查卡

M3 T2 第 2 课 · 损失函数

· 0. 这一课的主旨
· 1. ⚠⚠ 记号：ℓ 和 L 不是一回事
· 2. 回归损失：MSE 与 MAE
· 3. 分类损失
· 4. 把损失和概率、任务联系起来
· 5. ★★★ 输出–损失配对表（考纲逐条点名）
· 6. 从单样本损失到训练目标
· 7. ★★★ 为什么 sigmoid 和 BCE 是绝配（本课最该记的推导）
· 8. ★★ softmax + 交叉熵的学习信号
· 9. 总结自测
· ★ 速查卡

M3 T2 第 3 课 · 隐藏层怎么学（完整反向传播算例）

· 0. 这一课要回答的问题
· 1. 网络设定
· 2. 前向传播
· 3. ★★★ 反向：追一个隐藏权重到损失
· 4. ★★ 完成整步更新
· 5. 两道总结自测
· 6. 配套活动：M3 T2 ReLU by hand
· ★ 速查卡：隐藏层梯度四步

M3 T2 第 4 课 · 从一次更新到训练过程（批量 · epoch · 自动微分）

· 1. 一次更新只是开始
· 2. ★★ batch 和 epoch 是两个不同的单位
· 3. 后面例子用的固定数据集
· 4. ★★ 接到 Python：那个转置是最大的坑
· 5. ★★★ 自动微分做什么、不做什么（考纲明确要求）
· 6. ★★★ 那个必考的 TensorFlow 例子
· 7. 两道总结自测
· ★ 速查卡

M3 T2 第 1 课 • 激活函数

源页面：Activation function ｜ https://learn.adelaide.edu.au/courses/30362/pages/activation-function ｜ 整理日期：2026-10-14 ｜ 对应考纲：Topic 2「sigmoid、tanh、ReLU 及它们的导数；饱和与梯度行为；四种任务的输出层选择」

0. 这一课的主旨（页面开宗明义的一句）

> **把「激活函数对预测做了什么」和「它的导数对学习做了什么」分开。**
> 同时把**隐藏层的表示**和**任务要求的输出**分开。

这两组区分是整页的骨架，也是考点的来源。

1. ★★★ 关键区分：输出值 vs 导数

	作用在哪	干什么
激活值 a = g(z)	**前向**	往下一层传递
局部导数 g′(z)	**反向**	**乘在传回来的损失梯度上**

> ⚠⚠ **大的输出不代表大的梯度**
> （a large output need not imply a large gradient）。这两件事互不相干。

具体怎么乘：

> 传入的损失梯度 0.6 × 局部导数 0.25 = 传出去的梯度 0.15

> ★ **若局部导数为 0，这条路径对这个样本的梯度贡献就是 0。**

2. ★★★ 四个激活函数（必须会默写这张表）

函数	定义	导数	要点
Sigmoid	g(z) = 1/(1+e^(−z))	**g′ = g(1−g)**	映到 **(0,1)**；适合二分类输出概率；**会饱和**
tanh	g(z) = tanh z	**g′ = 1 − g²**	映到 **(−1,1)**；**输出范围关于 0 对称**；**也会饱和**
ReLU	g(z) = max(0, z)	**z<0 → 0；z>0 → 1**	高效，**正半轴不饱和**；**负输入拿到零梯度**
Leaky ReLU	g(z) = max(αz, z)	**z<0 → α；z>0 → 1**	**保留一点负侧梯度**，可减少死亡 ReLU

⚠⚠ 课程自己规定的约定（考试按这个判）

ReLU 和 Leaky ReLU 在 z = 0 处不可导，各家软件用不同约定。**本课规定**：

> $\mathrm{ReLU}'(0) = 0$
> $\mathrm{LeakyReLU}'(0) = 0.01$　　（当 $\alpha = 0.01$ 时）

> ★ α 是你自己选的一个小正数，**通常 0 < α < 1**，本课例子用 **α = 0.01**。

⚠ tanh 的两个坑

1. **它不是普通的正切函数。**双曲正切的定义是：

> tanh(z) = (e^z − e^(−z)) / (e^z + e^(−z))

用计算器的 **hyperbolic tangent** 键，或者用上面这个指数公式。

1. **tanh 的「输出范围关于零对称」≠「激活的实际均值为零」**——页面专门加了这句括注（the actual activation mean need not be zero）。

算例：tanh(1) ≈ **0.7616**，其导数 = 1 − 0.7616² ≈ **0.4200**。

> **⚠ 中间步骤多留几位，最后一步才四舍五入。**

3. ★★ 三个点上的数值表（最可能直接出计算题）

函数	z = −2	z = 0	z = 2
Sigmoid 输出	0.119	0.500	0.881
Sigmoid 导数	**0.105**	**0.250**	**0.105**
tanh 输出	−0.964	0	0.964
tanh 导数	**0.071**	**1**	**0.071**
ReLU 输出	0	0	2
ReLU 导数	**0**	**0** ← 课程约定	**1**
Leaky ReLU 输出（α=0.01）	**−0.02**	0	2
Leaky ReLU 导数	**0.01**	**0.01** ← 课程约定	**1**

★ 把 z = −2 这一列拆开看（页面专门讲解的一列）

> sigmoid：$a \approx 0.1192 \Rightarrow$ 导数 $= a(1-a) \approx 0.1050$
> ReLU：　输出 0　　⇒ 局部导数 0
> Leaky：　输出 −0.02　⇒ 导数 +0.01

> ⚠⚠ **注意 Leaky ReLU 这一格：输出是负的（−0.02），导数却是正的（+0.01）。**
>
> 页面原话：**输出的符号和斜率的符号是两回事。**

自测：z = −2、传入损失梯度 0.6 时，过 ReLU 和 Leaky ReLU 各传出多少?

> 过 ReLU：　$0.6 \times 0 = 0$　← 这条路径彻底断了
> 过 Leaky ReLU：　$0.6 \times 0.01 = 0.006$　← 很小，但没断

4. ★★ 饱和与「死亡 ReLU」

饱和（saturation）→ 梯度消失

sigmoid 和 tanh 的导数在输入绝对值很大时会变得非常小。 跨越很多层连乘这些小导数，就产生**梯度消失（vanishing gradient）**。

> **页面给的量级感**
> 五个 0.1 连乘 = **0.00001**，而这还没算上权重因子。

⚠⚠ **但页面特意加了限定**：

> **真实的梯度还包含那些权重、以及跨多条路径的求和。所以「激活斜率小」只是一个成因，不是完整的诊断。**

⇒ 考试里**别把梯度消失一股脑归给激活函数**，这个分寸要有。

死亡 ReLU（dead / dying ReLU）

一个 ReLU 单元可能变成**不激活**的：**如果它的预激活对所有相关的训练样本都是负的，它的梯度就是零，可能再也收不到有用的更新。**

⚠⚠ **页面给了一个很重要的限定**：

> **一个 ReLU 对某个观测不激活，对另一个观测可能是激活的。只有当它在相关的那批输入上一直不激活时，才叫它「死了」。**

⇒ **「某个样本上 ReLU 输出 0」≠「这个单元死了」。** 这是个很容易答错的点。

5. ★★ 非线性的概率输出，笔直的决策边界（T1 讲过，这里给了具体数字）

取 $w_1 = w_2 = 1$，$b = 0 \Rightarrow z = x_1 + x_2$

(x_1, x_2)	z	p = σ(z)	判类
(−1, −1)	−2	≈ **0.119**	0
(1, −1)	**0**	**0.500**	**1** ← 见下
(1, 1)	2	≈ **0.881**	1

> ⚠ **中间那个点正好落在边界上**
>
> ，按本课「**p ≥ 0.5 判类 1**」的约定，**判为类别 1。**

两种读图方式（页面用 A/B 两个面板对照）：

- **面板 A 当函数读**：每个输入对有一个概率，由曲面高度表示 ⇒ **这是一个弯曲的非线性曲面，不是平面。**
- **面板 B 当分类规则读**：等高线连接输出值相同的位置，**0.5 那条等高线就是类别改变的地方。**

接起来：σ(z) = 0.5 恰好在 z = 0 ⇒ 边界是 $x_1 + x_2 = 0$，**一条直线**——尽管概率曲面是弯的。

⚠⚠ 这里有一个措辞很讲究的点

> **限制来自「仿射分数 + 单调激活 + 固定阈值」这个组合，而不是单纯因为「模型只有一个神经元」。**

★ 所以**光把多个仿射层叠在最后那个 sigmoid 前面，直边界的限制依然在。**

⇒ **必须有非线性的隐藏激活**，输出分数才可能是「非线性中间特征的加权组合」，它对原始输入就不再是仿射的，**它的零等高线才可能弯曲或分成几段。**

> ⚠ 但页面也补了一句：**隐藏层的非线性让这种边界成为可能，并不保证随便哪组参数都能产生这样的边界。**

6. ★★★ 输出层怎么选（M3 考纲逐条点名的内容）

任务	输出层	说明
回归	**线性输出**	目标是无约束的实数
二分类	**一个 sigmoid**	表示 **P(y=1 | x)**
互斥多分类	**softmax**	**p_k = e^(z_k) / Σ_j e^(z_j)**，概率**加起来等于 1**
多标签（可同时成立）	**每个标签各一个 sigmoid**	⚠ **标签之间不必统计独立**

> **术语**
>
> **logit** 是过 sigmoid 或 softmax **之前**的原始分数，**不是概率。**
>
> K 个类别时，z_k 和 p_k 分别是类别 k 的分数和概率。

★ softmax 算例（要会手算）

三个类别的 logits **[1, 2, 0]**：

> 取指数：$[e^1, e^2, e^0] \approx [2.718, 7.389, 1]$
> 求和：$2.718 + 7.389 + 1 = 11.107$
> 归一化：$[0.245, 0.665, 0.090]$

> **第二类概率最大，三者加起来等于 1。**
> ✓

★★ 数值稳定技巧：先减去最大的 logit

> $[1, 2, 0] - 2 = [-1, 0, -2]$ ⇒ 归一化后概率完全相同

> **目的**
> 避免算出不必要的巨大指数。

> ★ 这也说明 **softmax 具有平移不变性——给每个 logit 都加 5，概率完全不变。**

⚠⚠ 三道必考的判断题

问题	答案
互斥分类任务里，**两个 softmax 概率能都等于 0.8 吗**?	**不能。**softmax 的概率加起来必须是 1，0.8+0.8 = 1.6 > 1
一张图既有猫又在户外，**两个 sigmoid 输出能都等于 0.8 吗**?	**能。**多标签的各个 sigmoid **各自独立输出，不必加起来等于 1**
给每个 logit 都加 5，softmax 概率变吗?	**不变**（平移不变性）

> ★ 对于「猫」和「户外」这种可共现的标签，**各自的 sigmoid 输出可以都很高，且不必加起来等于 1。**
>
> 共享的隐藏特征仍然可能捕捉到标签之间的关系。
>
> **关键区分是：互斥的类别 vs 可以同时出现的标签。**

7. 总结自测

Q：为什么说激活值、激活导数、损失是三个不同的量?

- **激活值**：前向传下去的那个数；
- **激活导数**：反向时乘在损失梯度上的**局部倍数**；
- **损失**：衡量预测与目标差距的那个标量。

★ 三者在计算图上处于不同位置，**大小之间没有必然联系。**

Q（页面的 Study check）：对每个函数，说出它的输出范围、在一个正输入和一个负输入处的导数、以及它适不适合做输出层的一个理由。

照第 2、3 节那两张表回答即可。★ **适不适合做输出层**的关键：

sigmoid 适合二分类（输出就是概率）；**ReLU 不适合**（输出无上界、且负半轴恒为 0）；

线性输出适合回归；**softmax 适合互斥多分类。**

★ 速查卡

导数： $\sigma' = \sigma(1-\sigma)$　　$\tanh' = 1 - \tanh^2$

ReLU′ = 0 (z<0), 1 (z>0), 本课约定 ReLU′(0) = 0

Leaky′ = α (z<0), 1 (z>0), 本课约定 Leaky′(0) = α = 0.01

反向： 传出梯度 = 传入梯度 × 局部导数　　导数为 0 ⇒ 这条路径断掉

输出层： 回归→线性　二分类→sigmoid

互斥多分类→softmax(和为1)　多标签→每标签一个 sigmoid(和不必为1)

softmax： p_k = e^(z_k) / Σ e^(z_j)　　★ 先减最大 logit，概率不变

M3 T2 第 2 课 • 损失函数

源页面：Loss function ｜ https://learn.adelaide.edu.au/courses/30362/pages/loss-function ｜ 整理日期：2026-10-14 ｜ 对应考纲：Topic 2「MSE 与交叉熵；回归/二分类/多分类/多标签的输出与损失选择」

0. 这一课的主旨

> 上一课决定**返回什么样的输出**，这一课决定**训练时怎么评判这个输出。**
>
> 要做三件事：**选一个和目标匹配的损失、把它算出来、解读它产生的学习信号。**

1. ⚠⚠ 记号：ℓ 和 L 不是一回事

记号	含义
$\ell_i = \ell(y_i, \hat{y}_i)$	**单个样本**的损失
$L = (1/N)\ \Sigma_i\ \ell_i$	在一个数据集或 mini-batch 上的**平均**经验损失

> **⚠ Topic 1 用 L 表示单个观测的损失**
>
> **；到了 Topic 2，那个量叫 ℓ_i，L 改成了平均值。**
>
> 跨 Topic 看材料时这里最容易混。

其它约定：

- **i 索引观测，k 索引类别**
- **所有 log 和 ln 都是自然对数**
- **K** = 互斥类别的个数
- 二分类里 **p（强调激活时写 a）是预测概率，不是阈值判出的类别**
- 二分类 sigmoid 输出时，**z 是原始分数（logit）**，**$p = \sigma(z)$**
- 多分类里 **y_{ik} 和 p_{ik}** 分别是观测 i 对类别 k 的目标指示和预测概率

2. 回归损失：MSE 与 MAE

损失	平均形式	解读
MSE 均方误差	$L = (1/N)\ \Sigma\ (y_i - \hat{y}_i)^2$	**平方 ⇒ 大误差被加倍重视**
MAE 平均绝对误差	$L = (1/N)\ \Sigma\ \lvert y_i - \hat{y}_i \rvert$	**线性增长**，对极端误差**不如 MSE 敏感**

★ 官方算例（要会算）

目标 **[3, −1]**，预测 **[2, 1]** ⇒ 残差（预测 − 目标）= **[−1, 2]**

> 平方： $[1, 4] \Rightarrow \text{MSE} = (1+4)/2 = 2.5$
> 取绝对值： $[1, 2] \Rightarrow \text{MAE} = (1+2)/2 = 1.5$

> ⚠ **平方误差不允许正负误差互相抵消。**
> ⚠ **单位不同**：MSE 的单位是**目标单位的平方**，MAE **保持目标的单位。**

自测：某个残差从 2 变成 4，两种贡献各怎么变?

> 平方误差贡献： $4 \rightarrow 16$ （×4，因为是平方）
> 绝对误差贡献： $2 \rightarrow 4$ （×2，线性）

⚠ 页面特意加的限定

> **MAE 在「需要对极端目标值稳健」时可能有用，但选择应当反映数据生成过程和误差的真实代价，而不是套用「某个损失总是更好」这种规则。**

★ 另外两条值得记：

- **一个残差 10，对平方误差贡献 100，对绝对误差贡献 10。**
- ⚠⚠ **MAE 能降低对极端残差的敏感度，但它修不了系统性的错误标签、缺失的人群、或者数据泄漏。**
- **MSE 和 MAE 估计的是目标的不同概括量**：分别对应**条件均值**和**条件中位数。**

3. 分类损失

二元交叉熵 BCE（$y \in \{0,1\}$，$p \in (0,1)$）

> $L_BCE = -(1/N)\ \Sigma_i\ [\ y_i \cdot \log p_i + (1-y_i) \cdot \log(1-p_i)\ ]$

多类交叉熵 CCE（K 个互斥类别，one-hot 目标 y_{ik}，softmax 概率 p_{ik}）

> $L_CCE = -(1/N)\ \Sigma_i\ \Sigma_k\ y_{ik} \cdot \log p_{ik}$

> ★ 因为**恰好只有一个目标分量是 1**，所以**单样本损失就是「给正确类别的那个概率」的负对数。**

★★ Sparse categorical cross-entropy（M3 考纲点名）

> **它用整数类别标签代替 one-hot 向量，但表示的是同一个计算。**

★ 官方算例

类别编号 **1, 2, 3**，真实类别是 **2** ⇒ one-hot 目标 **[0, 1, 0]**

概率 **[0.1, 0.7, 0.2]**：

> $\text{loss} = -(0 \cdot \ln 0.1 + 1 \cdot \ln 0.7 + 0 \cdot \ln 0.2) = -\ln 0.7 \approx 0.3567$

★ 只有「分配给观测到的那个类别」的概率进入这个 one-hot 损失。

⚠⚠ 整数标签的实现可能把同样三个类别编号成 0, 1, 2 —— 永远要核对映射关系。

4. 把损失和概率、任务联系起来

- **全部交叉熵计算都用自然对数 ln。**
- **★ 最小化「观测到的类别」的负对数概率，等价于最大化它的似然。**
- ★ 对照 M2：**GMM 是通过提高观测输入的概率密度来拟合的；分类器则是给「给定的目标类别」分配概率。**

★ 数值感（y = 0 的情形，容易算反）

情形	正确类别的概率	损失
y = 0，p = 0.8	**0.2**（因为正确类是 0）	−ln(0.2) ≈ **1.609**
y = 0，p = 0.1	**0.9**	−ln(0.9) ≈ **0.105**

★ 有把握但判错，会受到很大的惩罚。

⚠⚠ 损失 ≠ 准确率（考点）

损失是一个优化目标；准确率数的是阈值判定之后的决策。
两个分类器可能做出完全相同的硬判定，却有不同的交叉熵——因为它们的概率不同。

⚠ 只有在任务、单位、聚合约定都可比时，才能比较损失数值。

5. ★★★ 输出–损失配对表（考纲逐条点名）

任务	输出	常用损失
连续回归	线性	**MSE 或 MAE**
二分类	一个 sigmoid 概率	**二元交叉熵**
互斥多类	softmax 概率	**多类交叉熵**
可共现的多标签	每标签一个 sigmoid	**每标签各做一次 BCE**

⚠⚠ 一个极易踩的工程坑

> **软件里数值稳定的损失函数常常直接接收原始 logits，内部自己做 sigmoid 或 softmax。**
>
> **用这种选项时，不要再把激活函数套第二遍。**

自测：四个互斥类别，标签存成整数 0–3，用什么输出和损失?

> **softmax 输出 + sparse categorical cross-entropy。**
>
> （整数标签 ⇒ 用 sparse 版；互斥 ⇒ 用 softmax。）

6. 从单样本损失到训练目标

★ 算例

目标 **[1, 0]**，概率 **[0.8, 0.3]**：

> 第一个（$y=1, p=0.8$）： $-\ln(0.8) \approx 0.2231$
> 第二个（$y=0, p=0.3$）： 正确类概率 $= 0.7 \Rightarrow -\ln(0.7) \approx 0.3567$
> ↑注意这里要用 $1-p$
>
> 和 $= 0.5798$
> 均值 $= 0.2899$

> **★ 平均梯度 = 各个单样本梯度之和 ÷ 样本数。**

> **⚠⚠ 求导之前，先说清楚这道题用的是「和」、「平均」还是「半平方误差」。**

⚠⚠ 两个不同的导数（最容易混的一处）

> 对概率求导： $\partial \ell / \partial p = -y/p + (1-y)/(1-p)$
> 对 logit 求导：$\partial \ell / \partial z = p - y$ ← 已经包含了 sigmoid 的链式因子

> ★ 后者就是 T1 学的那个 **δ = p − y。两者不是一回事，别张冠李戴。**

数值保护与校准

- **ε（epsilon）** 是一个很小的正数（如 10^{-7}），用来让概率**远离恰好的 0 和 1。**
- **ln(0) 无定义。** 手算时用严格介于 0 和 1 之间的概率；教学代码里**裁剪到 [ε, 1−ε]** 可以避免无穷大。
- ⚠⚠ **裁剪只是改变了极端的数值，不能被误当成「模型校准良好」的证据。**

- **校准（calibration）的定义**：**预测概率与观测频率一致**——在被赋予概率 0.8 的那些案例里，**大约 80% 应该是正例。**

7. ★★★ 为什么 sigmoid 和 BCE 是绝配（本课最该记的推导）

设 **a = p = σ(z)**，y 是二元目标。

⚠ 先说清楚「半平方误差」不是 MSE

> ℓ_half-square $= \frac{1}{2}(a-y)^2 \quad \Rightarrow$ 对 a 的导数 $= (a-y)$

> **⚠ 这不是上面那个 MSE 公式**
>
> 普通平方误差求导给的是 **2(a−y)**，而在 N 个观测上取平均还会多一个 **1/N**。
>
> **下面的式子用的是「半平方」这个约定。**

两种搭配对 logit 的导数

> sigmoid + 半平方误差：$\partial\ell/\partial z = (a-y)\cdot a(1-a)$
> 　　└─ 多出来的这个因子是祸根
>
> sigmoid + BCE：$\partial\ell/\partial z = a-y$ 　★ 干净利落

> **★★ sigmoid 饱和时，a(1−a) 非常小。**
>
> **⇒ 对一个「有把握但判错」的二分类预测，BCE 给出的纠正梯度比 MSE 强得多。**

★ 官方算例（y = 1，a = 0.01，即非常自信地判错了）

> 半平方：$(0.01-1)\times 0.01\times 0.99 = (-0.99)(0.0099) = -0.009801$
> BCE：$0.01-1 = -0.99$

> **两者的符号都主张抬高 logit，但半平方的梯度小了整整两个数量级**
> **——因为 sigmoid 的局部斜率太小。**

⚠ **页面的限定**：

> **这个优势只针对「这一对输出–损失的组合」。BCE 并不能阻止深层网络中别处出现小梯度。**

自测：y = 1、p = 0.2 时，p − y 是哪个导数?

是对 logit 的导数

$\partial \ell/\partial z = 0.2 - 1 = -0.8$。

链式法则把它和「对概率的导数」连起来：**$\partial \ell/\partial z = (\partial \ell/\partial p)\cdot(\partial p/\partial z)$**，其中 **$\partial p/\partial z = p(1-p)$**。

8. ★★ softmax + 交叉熵的学习信号

$$p_k = \exp(z_k) / \Sigma_j \exp(z_j) \qquad \ell = -\Sigma_k y_k \ln(p_k)$$

$$\Rightarrow \partial \ell/\partial z_k = p_k - y_k$$

★ 这是**对 logit 的导数，不是对概率、也不是对权重的导数。**

页面说：**可以直接使用这个结果，不必推导完整的 softmax 雅可比矩阵。**

三步操作法

1. 把 one-hot 目标**按和概率相同的类别顺序**写出来
2. **用概率向量减去目标向量**
3. **解读符号**：**正确类别那一项是负的** ⇒ 梯度下降会**抬高**那个 logit；**其余项是正的** ⇒ 它们的 logit 被**压低**

★ 官方算例

p = **(0.1, 0.7, 0.2)**，真实类别是 **2**（类别编号 1,2,3）：

$$y = (0, 1, 0)$$
$$\text{loss} = -\ln(0.7) \approx 0.357$$
$$\text{logit 梯度} = p - y = (0.1, -0.3, 0.2)$$

★★ 这三项加起来等于零。

这是个很好的自查。

★ 正确类概率趋近 1 时梯度趋近 0；有把握地判错则给出很大的纠正信号。

⚠ 别忘了这只是中间量

网络真正更新的是共享的权重和偏置。

要用链式法则从这些 logit 导数得到它们的梯度；

改动一个权重可能影响好几个 logit 和好几个观测。

对 N 个样本的平均损失，就把各样本的参数梯度取平均。

★ 这把二分类的 **p − y** 推广到了互斥多类。

9. 总结自测

Q：一个分类器降低了训练 BCE，但没有改变任何类别判定。它学到东西了吗？在新数据上有用吗？

> **学到了**
> ——它改进了**概率估计**（损失对概率敏感，而准确率只看阈值后的判定，所以准确率没动很正常）。

> **⚠ 但这不能证明它在新数据上有用**
> 这只是训练集上的改进，**泛化要靠验证/测试证据**。

Q（页面的 Study check）：给一段任务描述，说出目标的表示方式、输出激活、以及兼容的损失；再判断给出的公式是单样本损失还是平均损失。

> 看公式前面有没有 **(1/N) Σᵢ** —— 有就是平均损失，没有就是单样本。

★ 速查卡

ℓ_i = 单样本损失　　$L = (1/N)\Sigma\ell_i$ = 平均损失　　⚠ T1 的 L 是这里的 ℓ

回归：　$MSE = (1/N)\Sigma(y-\hat{y})^2$　　$MAE = (1/N)\Sigma|y-\hat{y}|$
　⚠ MSE 单位是目标单位的平方；MAE 保持原单位
　⚠ MSE→条件均值　MAE→条件中位数

分类：　$BCE = -(1/N)\Sigma[y\cdot\ln p + (1-y)\cdot\ln(1-p)]$
　$CCE = -(1/N)\Sigma\Sigma\ y_ik\cdot\ln\ p_ik$　⇒ 单样本 = −ln(正确类的概率)
　sparse 版 = 整数标签，同一个计算

配对：　回归→线性+MSE/MAE　二分类→sigmoid+BCE
　互斥多类→softmax+CCE　多标签→每标签 sigmoid+BCE
　⚠ 软件的 from_logits 选项会内部做激活，别套第二遍

梯度：　$\partial\ell/\partial p = -y/p + (1-y)/(1-p)$　← 对概率
　$\partial\ell/\partial z = p - y$　← 对 logit（含 sigmoid 链式因子）
　softmax+CE：　$\partial\ell/\partial z_k = p_k - y_k$　★ 各项和为 0

★ sigmoid+半平方：　$\partial\ell/\partial z = (a-y)\cdot a(1-a)$　← 饱和时几乎归零
　sigmoid+BCE：　$\partial\ell/\partial z = a-y$　← 不受饱和拖累
　算例 y=1,a=0.01：　−0.009801 vs −0.99

M3 T2 第 3 课 • 隐藏层怎么学（完整反向传播算例）

源页面：How a hidden layer learns + M3 T2 ReLU by hand ｜ 整理日期：2026-10-14 ｜ 对应考纲：Topic 2「部分隐藏层权重的梯度」 ｜ ★★ **M3 T2 Tutorial Q3 考的就是这个**（官方描述：一个隐藏层权重的梯度、自动微分、batch/epoch 计数）

0. 这一课要回答的问题

> **隐藏神经元没有自己的观测目标，那怎么决定该怎样改它的权重?**

答案：追踪它对最终损失的影响。 隐藏权重改变隐藏分数 → 改变激活 → 改变输出分数 → 最终改变损失。

链式法则把这串效应连起来，所以隐藏权重可以用和输出神经元同一个最终目标来训练。

1. 网络设定

⚠ **这是一组全新的起始参数，不是 T1 那次更新之后的参数。**

观测仍是 **x = [1, 2]**，目标 **y = 1**。

单元	输入权重	偏置	激活
隐藏 1	[0.2, 0.4]	0.1	**ReLU**
隐藏 2	[−0.5, 0.3]	−0.2	**ReLU**
输出（输入是 h_1, h_2）	[0.7, −0.4]	−0.1	**Sigmoid**

> ★ 隐藏 1 用的是 T1 那套权重偏置，**但激活换成了 ReLU。**

记号

- **h_1, h_2** = 隐藏输出；**z_1, z_2** = 它们过 ReLU 之前的分数
- **s** = 最终分数（特意用 s 以区别于隐藏分数）；**p = σ(s)**
- **L** = 这一个观测的 BCE 损失 ⚠ **等同于损失那一课里的 ℓ**
- ⚠ **W 和 b 的下标表示「第几层」，不是幂次**：W_1/b_1 属隐藏层，W_2 属输出层
- **w_ji** 连接输入特征 i 到隐藏单元 j；**W_2 = [v_1, v_2]**，**c** 是输出偏置

⚠ ReLU 在 0 处的约定（再强调一次）

> ReLU′ = 1（z>0）、0（z<0）。**在恰好为零处数学导数无定义，本课的计算和 notebook 一律用 0。**

2. 前向传播

步骤	计算	含义
隐藏 1	$z_1 = 0.2(1) + 0.4(2) + 0.1 = \mathbf{1.1} \Rightarrow h_1 = \mathbf{1.1}$	**对这个观测是激活的**
隐藏 2	$z_2 = -0.5(1) + 0.3(2) - 0.2 = \mathbf{-0.1} \Rightarrow h_2 = \mathbf{0}$	**对这个观测不激活**
输出分数	$s = 0.7(1.1) - 0.4(0) - 0.1 = \mathbf{0.67}$	组合两个隐藏激活
预测与损失	$p = \sigma(0.67) = \mathbf{0.661503}$；$L = -\log p = \mathbf{0.413241}$	与 $y = 1$ 比较

> ⚠ **ReLU 不会把隐藏值变成概率——h_1 完全可以大于 1**（这里就是 1.1）。

两道自测

问题	答案
$h_2 = 0$，若只把它的输出权重从 −0.4 改成 −0.3，p 会变吗?	**不会。**因为 **$h_2 = 0$**，乘以任何权重都是 0
$h_2 = 0$ 是否意味着隐藏单元 2 永远帮不上忙?	**不是。⚠ 「在这个观测上不激活」≠「永远不激活」**

3. ★★★ 反向：追一个隐藏权重到损失

挑 **从 x_2 到隐藏单元 1 的那个权重 0.4**，记作 **w_{12}**

> **（行 1 选隐藏单元 1，列 2 选输入 2）**

路径：$w_{12} \to z_1 \to h_1 \to s \to p \to L$

四步，每步回答一个更小的问题

Step 1 从输出分数起步（sigmoid + BCE 的化简，和 T1 一样）

$\partial L/\partial s = p - y = 0.661503 - 1 = -0.338497$

Step 2 走到隐藏激活 h_1

$s = 0.7h_1 - 0.4h_2 - 0.1 \Rightarrow \partial s/\partial h_1 = 0.7$ ← 就是那个输出权重

$\partial L/\partial h_1 = (-0.338497)(0.7) = -0.236948$

Step 3 穿过 ReLU

$z_1 = 1.1 > 0 \Rightarrow$ 局部导数 $= 1 \Rightarrow$ 原样通过

$\partial L/\partial z_1 = (-0.236948)(1) = -0.236948$

⚠ 隐藏单元 2 此刻分数为负，它那边的导数是 0

Step 4 到达选定的权重

$\partial z_1/\partial w_{12} = x_2 = 2$

$\partial L/\partial w_{12} = (-0.236948)(2) = -0.473896$

更新这一个权重（η = 0.1）

$w_{12},\text{new} = 0.4 - 0.1 \times (-0.473896) = 0.447390$

梯度为负 ⇒ 在局部增大这个权重有利于这个正例。

★ 自测：为什么隐藏层的梯度里会出现输出权重 0.7?

因为**隐藏激活对损失的影响必须穿过输出层。**

0.7 正是 s 对 h_1 的局部导数——没有它，这条链就断了。

⇒ **这也解释了 L11 片子里那句「上层权重会出现在下层的梯度里」。**

4. ★★ 完成整步更新

全部梯度（都用原始参数算）

输出权重梯度 $= (p - y)\cdot[h_1, h_2] = (-0.338497)\cdot[1.1, 0] = [-0.372347, 0]$

输出偏置梯度 $= p - y \quad = -0.338497$

隐藏 1 的输入权重梯度 $= [-0.236948, -0.473896]$

隐藏 1 的偏置梯度 $\quad = -0.236948$

隐藏 2 的输入权重梯度 $= [0, 0]$ ← 因为它这次不激活

⚠⚠ **先用原始的输出权重把这些全部算完，然后所有参数一起更新。**

一步之后（η = 0.1）

参数	更新后
隐藏 1 权重与偏置	**[0.223695, 0.447390]；0.123695**
隐藏 2 权重与偏置	**[−0.5, 0.3]；−0.2 ← 完全没变**
输出权重与偏置	**[0.737235, −0.4]；−0.066150**
新的概率与损失	**p = 0.700487；L = 0.355979**（原为 p=0.661503；L=0.413241）

★ 注意两处「没变」：

- **隐藏 2 整个没动**——它这次不激活，梯度全是 0；
- **输出的第二个权重 −0.4 也没动**——因为它的梯度 = $(p-y)\cdot h_2 = (p-y)\cdot 0 = 0$。

验算（要会复现）

隐藏 1 新分数 $= 0.223695(1) + 0.447390(2) + 0.123695 \approx 1.242169$ （仍为正，ReLU 照常通过）
新输出分数 $s \approx 0.849619$
新概率 $p = \sigma(0.849619) \approx 0.700487$

⚠ **用未四舍五入的值计算才准确。**

⚠ **只有重新跑一次完整的前向，才能确认「改变全部参数」的合并效果。**

⚠ **这个改进只针对这一个样本，不是泛化的证据。**

★★ 自测：为什么隐藏层梯度必须用原来的 0.7，而不是更新后的 0.737235?

因为**所有梯度都要在同一个参数状态上求值**——也就是**做前向传播时的那个状态。**

⚠ 若中途换用新值，算的就不是这一点处的梯度了，**更新方向会失真。**

★ 这和 T1「三个梯度全部算完再一起更新」是同一条纪律。

5. 两道总结自测

Q：隐藏权重没有自己的类别标签，怎么学?

不需要单独的目标。

它通过「隐藏分数 → 激活 → 输出分数 → 损失」这条链影响最终损失，

链式法则把这串效应连起来，所以可以**用和输出神经元同一个最终观测目标**来训练它。

Q：反向传播和梯度下降是两套不同的更新规则吗?

不是

，它们是**两件不同的工作**：**反向传播负责算梯度，梯度下降负责用梯度改参数。**

（这条在 T1 第 2 课已经强调过，是反复考的点。）

6. 配套活动：M3 T2 ReLU by hand

目的

看简单的非线性隐藏单元怎样**组合出一个新形状。**

⚠ 这是一个**回归式**的小构造、**线性输出**，**不是类别概率模型。**

输出特意叫 r，以免和损失那课里的观测目标 y 混淆。

结构

两个并联的 ReLU 单元，输出相减。两个单元都直接从同一个 x 取输入。

两个输入权重都 $= 1$
隐藏偏置分别是 0 和 -2
输出权重分别是 $+1$ 和 -1

⚠⚠ A 不喂给 B

——**每一支都直接从 x 算**，两个结果再一起进入减法。

⚠ **A 和 B 是算出来的隐藏激活，不是训练得到的参数**（这些值是为了演示表示能力而挑的）。

三段计算

$$A = \mathrm{ReLU}(x) \qquad B = \mathrm{ReLU}(x-2) \qquad r = A - B$$

$x < 0$ ： $A = 0,\ B = 0,\ r = 0$
$0 \le x < 2$ ： $A = x,\ B = 0,\ r = x$
$x \ge 2$ ： $A = x,\ B = x-2,\ r = x-(x-2) = 2$

写成分段函数

$$r(x) = \begin{cases} 0 & x < 0 \\ x & 0 \le x < 2 \\ 2 & x \ge 2 \end{cases}$$

画出来是一条「先平、再上升、再变平」的折线，折点在 x = 0 和 x = 2。

★ 自测：x > 2 之后，两个隐藏输出都还在涨，为什么最终输出不涨了？

因为**它们以完全相同的速率上涨，而又被相减——差值因此是常数。**

★ 这就是**用 ReLU 搭出「饱和平台」的办法**：两条同斜率的斜坡相减。

⚠ 自测：这个构造是否证明「训练一定能发现这组权重」？

不能。

页面明说这些值是**为了演示表示能力而挑选的，不是训练实验得来的。**

⇒ **「网络能表示某个函数」和「训练能找到它」是两件事**——这条在 T1 也强调过。

★ 速查卡：隐藏层梯度四步

① $\partial L/\partial s = p - y$ （sigmoid + BCE）
② $\partial L/\partial h_j = (\partial L/\partial s) \times v_j$ （乘上该隐藏单元的输出权重）
③ $\partial L/\partial z_j = (\partial L/\partial h_j) \times g'(z_j)$ （穿过激活；ReLU 时是 1 或 0）
④ $\partial L/\partial w_{ji} = (\partial L/\partial z_j) \times x_i$ （乘上该权重对应的输入）
$\partial L/\partial b_j = \partial L/\partial z_j$

⚠ 全部梯度用「同一个（原始）参数状态」算完，再一起更新
⚠ 某隐藏单元不激活 ⇒ 它的输入权重梯度全为 0，该单元这一步完全不动
⚠ $h=0$ 的那个输出权重，梯度 $= (p-y)\cdot h = 0$，也不动

M3 T2 第 4 课 · 从一次更新到训练过程（批量 · epoch · 自动微分）

源页面：From one update to a training process ｜ 整理日期：2026-10-14 ｜ 对应考纲：Topic 2「batch、epoch、自动微分」 ｜ ★★★ **考纲专门点名这一页**：原文要求"study the required, explained TensorFlow example in ｜ *From one update to a training process*"，并且**要能解释 tape 记录了什么、gradient 返回什么、 ｜ 哪个操作真正改变参数。**

1. 一次更新只是开始

训练算法反复做：**选样本 → 算预测和损失 → 算新的梯度 → 更新参数 → 重复。**

> ★ **权重会变，观测到的训练标签不会变。**

> ⚠⚠ **训练损失下降只说明模型在拟合训练目标，不一定说明它能预测没见过的数据。**

★★ 四个阶段与各自的典型错误（很像考点清单）

阶段	做什么	典型错误
Forward	用**当前**参数算激活、预测、损失	**复用更新之前的激活值**
Backward	用**这一次前向**的结果算出全部损失梯度	**在算隐藏层梯度之前就更新了输出权重**
Update	减去「学习率 × 梯度」	**把梯度加上去**，或步子过大
Repeat	用新参数做一次新的前向	**每一步都复用旧梯度**

> ★ 第二行那个错误正是上一课强调的：**必须用原始参数把全部梯度算完再一起更新。**

2. ★★ batch 和 epoch 是两个不同的单位

术语	定义
batch（批）	**用来计算一次更新的那组观测**
mini-batch	训练集的一个**子集**
epoch（轮）	**把训练集完整过一遍**

> $L_{batch} = (1/N)\,\Sigma_n\, L_n$　　N = 当前批里的观测数

> ⚠ Topic 2 把单个损失叫 ℓ_n，这里写作 L_n，**指的是同一个量。**

★ 必考的计数题

> **100 个训练观测、批大小 20，每个观测每轮用一次**
>
> ⇒
>
> **一个 epoch 有 5 次更新**（100 ÷ 20）；**三个 epoch 就是 15 次更新。**

> **全批量梯度下降（full-batch）**
>
> 100 个全用上，**每个 epoch 只更新一次。**

⚠⚠ 平均只做一次

> **用平均损失时，每个样本贡献的是「它的梯度 ÷ N」。**
>
> **这个平均只做一次，不要在每一层再除一遍。**

sigmoid/BCE 的输出分数梯度数组是 `(P - Y)/N`。

（P 是预测概率数组，Y 是二元目标数组；每一项都是**平均 BCE 对那个观测输出分数的导数。**）

★ 一个很有意思的数值例子

> 两个样本对某参数的梯度分别是 **−0.2** 和 **+0.6**，**平均是 +0.2** ⇒ **梯度下降会减小这个参数。**

> **⚠⚠ 一次基于平均损失的更新，可能在改善平均的同时让某一个观测变差。**

3. 后面例子用的固定数据集

四个观测:

> $([\,1,\ 2], y{=}1)$　$([-1, -2], y{=}1)$
> $([\,2, -1], y{=}0)$　$([-2,\ 1], y{=}0)$

> **⚠ 这是刻意做小的教学数据，不是泛化能力的检验。**
>
> **⚠ 它和第 1 课那个「正方形四角」的图示不是同一组数据。**

参数从「隐藏层怎么学」那一课的初始值重新开始（隐藏1 [0.2,0.4] 偏置 0.1；隐藏2 [−0.5,0.3] 偏置 −0.2；输出 [0.7,−0.4] 偏置 −0.1；隐藏 ReLU、输出 sigmoid）。

> **★ 这四个观测上的平均 BCE 起始值是 0.580352。**

4. ★★ 接到 Python：那个转置是最大的坑

> ⚠⚠ **手推时把一个输入存成「列」；代码里把观测存成「行」。**
>
> ⇒ **代码里每个权重矩阵都是手推版本的转置**：`W1 = W1ᵀ`，`W2 = W2ᵀ`，`b2` 装的是输出偏置 c。
>
> **转置只是行列互换，不改变网络本身。**

量	形状与含义
`X`; `Y`	**(N,2)** 输入；**(N,1)** 观测二元目标
`W1`; `b1`	**(2,2)** 隐藏权重；**(1,2)** 隐藏偏置。★ **代码 W1 的每一列属于一个接收的隐藏单元**
`Z`; `H`	**(N,2)** 隐藏预激活与 ReLU 激活
`W2`; `b2`	**(2,1)** 输出权重 v_1,v_2；**(1,1)** 输出偏置 c
`S`; `P`	**(N,1)** 输出 logits 与 sigmoid 概率

⚠ 这里有个会掩盖问题的巧合

> **本例 W1 是方阵，所以转置不改变形状——但确实挪动了元素。**
>
> **换成 3 个隐藏单元、2 个输入特征**：**手推矩阵是 3×2，代码矩阵是 2×3**，输出权重的代码矩阵是 **3×1**。

★ 读一列来验证

代码 `W1` 的第一列是 [0.2, 0.4] —— 隐藏单元 1 接收 x_1、x_2 的那两个权重。

对输入行 `[1, 2]`：`1×0.2 + 2×0.4 = 1.0`，加偏置 `0.1` ⇒ **1.1** ✓ 和手算一致。

第二列 `[-0.5, 0.3]` 给出另一个隐藏分数。

第一行的结果：`Z=[1.1, -0.1]`，`H=[1.1, 0]`，`S=[0.67]`，`P≈[0.661503]` —— **和上一课完全吻合**，四行的平均损失 **0.580352**。

代码里的三个要点

- `@` **是矩阵乘法**
- **把一个「行形状」的偏置加到每一行上，靠的是 NumPy 广播**
- ★ **损失表达式直接从 logits 算 BCE，为的是避免对四舍五入过的概率取对数**

自测：N 个观测、2 个输入特征、3 个隐藏单元，H / P / Y 的形状?

> H 是 (N, 3)　← 行数 = 观测数，列数 = 隐藏单元数
> P 是 (N, 1)　← 每个观测一个概率
> Y 是 (N, 1)　← 每个观测一个目标

5. ★★★ 自动微分做什么、不做什么（考纲明确要求）

> **自动微分追踪一次前向计算里的各个运算，并应用链式法则求导数。**
> **它把你练过的那些算术自动化了。**

⚠⚠ 它不做这些（这句最可能出选择题）

> **它不会替你选一个有用的目标，不会修复数据泄漏，也不保证学习率是好的。**
> **参数更新是由优化器完成的。**

6. ★★★ 那个必考的 TensorFlow 例子

⚠ 它刻意换了设定：一个标量输入 2.0、一个 ReLU 神经元、平方误差损失、目标 5.0。

这不是前面那个二分类批量——链式法则的机制相同，但损失的导数不同。

```
w = tf.Variable(1.5)
b = tf.Variable(-1.0)
with tf.GradientTape() as tape:
    z = w * 2.0 + b
    a = tf.nn.relu(z)
    loss = 0.5 * (a - 5.0)**2
gw, gb = tape.gradient(loss, [w, b])
w.assign_sub(0.05 * gw)
b.assign_sub(0.05 * gb)
```

★★ 三个角色（考纲原文要求能解释的三件事）

问题	答案
tape 记录了什么?	`tf.Variable` **标记哪些量是可训练的；tape 在前向计算期间记录「涉及这些变量的运算」**
`tape.gradient` **返回什么?**	**它应用链式法则，按请求的顺序** `[w, b]` **返回梯度 −6 和 −3。⚠⚠ 它不更新参数**
哪个操作真正改变参数?	`assign_sub` —— 它执行随后的减法

★ 把数字追一遍（要会复现）

前向： $z = 1.5 \times 2.0 + (-1.0) = 2$

$a = \mathrm{ReLU}(2) = 2$

$\mathrm{loss} = 0.5 \times (2-5)^2 = 0.5 \times 9 = 4.5$

反向： $\partial \mathrm{loss}/\partial a = a - 5 = -3$ （$0.5(a-5)^2$ 的导数）

$z = 2 > 0 \Rightarrow$ ReLU 贡献倍数 1

$\partial \mathrm{loss}/\partial w = (-3) \times 1 \times 2.0 = -6$ ← 乘输入 2.0

$\partial \mathrm{loss}/\partial b = (-3) \times 1 \times 1 = -3$ ← 偏置的倍数是 1

更新（$\eta = 0.05$）：

$w = 1.5 - 0.05 \times (-6) = 1.8$

$b = -1.0 - 0.05 \times (-3) = -0.85$

重新前向： $z = 1.8 \times 2 + (-0.85) = 2.75 = a$

$\mathrm{loss} = 0.5 \times (2.75-5)^2 = 2.53125$ ← 从 4.5 降下来了

★★ 两个梯度都是在任何一次更新之前算出来的

—— 又是那条纪律。

⚠ 自测：如果把两行 `assign_sub` 删掉，调用 `tape.gradient` 还能训练这个模型吗?

不能。

`tape.gradient` 只负责计算梯度，没有任何东西去改变参数。

★ 这正是「**反向传播算梯度、优化器改参数**」那条区分在代码里的体现。

★ 范围说明

PyTorch 那段对照是可选的

，外部框架教程也是可选的。

M3T1 的 notebook 用的是 NumPy。

不要求记 TensorFlow 语法、不要求手写、也不要求安装。

7. 两道总结自测

Q1：解释完整的训练循环，以及为什么每次更新都需要重新计算。

输入 → 预测 → 损失 → 梯度 → 更新 → 新的前向。

因为参数变了，所有依赖参数的量（激活、预测、损失、梯度）全都过期了。

⚠ 复用旧激活或旧梯度，算出来的就不是当前这一点处的下降方向。

Q2（指向 Topic 3）：什么证据能区分「拟合了训练数据」和「学到了有用的模型」？

训练损失下降只证明前者。

要区分，必须看**留出数据上的证据**——

验证损失、训练与验证曲线的分叉，这正是 Topic 3 的内容。

★ 速查卡

batch ＝算一次更新所用的那组观测
epoch ＝把训练集完整过一遍
100 个观测 / 批大小 20 ⇒ 1 epoch = 5 次更新，3 epoch = 15 次

平均损失： $L_batch = (1/N)\Sigma L_n$　梯度数组 $= (P - Y)/N$
⚠ 这个 1/N 只除一次，别在每层再除一遍
⚠ 平均改善 ≠ 每个样本都改善

代码 vs 手推： 代码矩阵 = 手推矩阵的转置（代码按「行=观测」存）
代码 W1 的每一列 = 一个接收的隐藏单元
3 隐藏单元 2 特征 ⇒ 手推 3×2，代码 2×3

自动微分： tape 记录涉及 tf.Variable 的前向运算
tape.gradient 用链式法则返回梯度 —— ⚠ 它不更新参数
assign_sub 才是真正改变参数的那一步
⚠ 它不替你选目标、不修数据泄漏、不保证学习率好